

ICT1011: Computer Organisation and Architecture

Tutorial 6: The Stack and Modular Programming

Q1. Stack Manipulation Operations

Fig. Q1 shows the contents on the system stack and the current position of the system stack pointer (**SP**). With reference to Fig. Q1, answer the following questions.

Note: Your answers must not make any references to the absolute addresses of the stack shown.

- (a) What is the current content in the register **R1**?
- (b) Give the instruction to push the content in **R10** onto the system stack.
- (c) Give a single instruction to pop off four word-sized items from the stack.
- (d) Give an instruction to retrieve the 3rd word-sized data (i.e. **0x0000**) on the stack and place it into **R10**.

	Address	Contents
(SP) →	0x30E4	0xFFFFE
	0x30E6	0xAD23
	0x30E8	0x0000
	0x30EA	0x0820
	0x30EC	0x1234

Figure Q1 – The system stack and its contents

Q2. Modular Programming – Subroutine Call and Parameter Passing

Fig. Q2a shows a code segment for subroutine **Sub1** and the calling program that makes use of it. With reference to Fig. Q2a, answer the following questions:

- (a) With reference to the instruction **call #Sub1**, answer the following questions:
 - i) What addressing mode is used by this instruction?
 - ii) With the aid of the disassembled memory contents shown in Fig. Q2b, determine the address where the subroutine **Sub1** is stored.
- (b) How are the parameters passed to the **Sub1**, is it by value or by reference?
- (c) With the aid of the disassembled memory contents shown in Fig. Q2b, give the hexadecimal content of the **PC** and **SP** immediately after the execution of each of the instructions at (b1), (b2) and (b3) shown in Fig. Q2b. Assume execution begins at instruction (a).
- (d) What is the 16-bit hexadecimal content in **R10** after instruction (c) in Fig. Q2a is executed?
- (e) Give the instruction at (d) that would remove the two parameters pushed to the stack?

- (f) The partially completed subroutine **Sub1** implements the function **Y = 2X**, where the parameter **X** passed to the subroutine is a signed word-sized number. Complete the subroutine **Sub1** using the help of the various comments provided in Fig Q2a

```

.data
X      .word    9
Y      .space   1

.text
Start:  mov.w    #3100h,SP      ; (a) Initialize stack pointer
        mov.w    #X,R10
        push.w   R10           ; (b1)
        mov.w    #Y,R10
        push.w   R10           ; (b2)
        call     #Sub1         ; (b3)
        mov.w    -2(SP),R10    ; (c)
        ?        ?            ; (d) Remove the two stack parameters
        :        :

Sub1:   ?        ?            ; Save used registers R10,R11,R12
        mov.l    ?(SP),R10     ; Retrieve address of X from stack
        mov.l    ?(SP),R11     ; Retrieve address of Y from stack
        :        :            ; Retrieve the value of X into R12
        :        :            ; Compute the value of 2X into R12
        :        :            ; Move result directly to memory variable Y
        ?        ?            ; Restore used registers R10, R11, R12
        ret                               ; Return to calling program
        .end

```

Figure Q2a – A partially completed assembly language program to demonstrate subroutine call

<u>Addr</u>	<u>Machine Code</u>	<u>Instruction</u>	
004400:	4031 3100	MOV.W #0x3100,R1	
004404:	40B2 5A80 015C	MOV.W #0x5a80,&Watchdog_Timer_WDTCTL	
00440a:	403A 2400	MOV.W #0x2400,R10	
00440e:	120A	PUSH R10	← (b1)
004410:	403A 2402	MOV.W #0x2402,R10	
004414:	120A	PUSH R10	← (b2)
004416:	12B0 4434	CALL #Sub1	← (b3)
00441a:	411A FFFE	MOV.W 0xffff(SP),R10	

Figure Q2b – Disassembled contents of memory in code area